Diagnose Before Solving

Case study: an AI conversation simulation, rebuilt so it can't fail in front of the people it's for

David Bacon · Instructional design, assessment design, AI implementation
Draft for review, November 2026

Try it	Simulation: diagnose-before-solving.vercel.app · Lead-in training: dbacon6093.github.io/diagnose-before-solving
Learner	An internal tech support specialist taking an angry call
Format	A short web-based lead-in, then a live text "call" with an AI character, then scored, evidence-based feedback
Built with	Claude Sonnet 5 (the character), Claude Opus 5 (the assessor), Next.js on Vercel, Upstash Redis, plain HTML, CSS and JavaScript for the training
Timeline	August to November 2026
My role	Everything: needs and scope, learning and assessment design, prompt design, test design, the training build, and directing an AI coding assistant through the application build

The problem

The first version of this simulation ran on a third-party platform. When a prospective employer tried to open it, the account's credits had run out and nothing loaded. A portfolio piece that fails in front of a hiring manager is worse than no portfolio piece.

So the brief I set myself had two halves. Keep the learning design, which was sound. And rebuild everything underneath it so that a visitor **always** sees a working, complete artifact, whatever happens to the budget, the API, or the database.

The learning problem

The skill is one every support team needs and few train well: **diagnose before you solve.** The learner takes a call from Marcus, a regional sales lead whose team's numbers have dropped since a CRM update. He is angry and sure the new required field is broken.

It isn't broken. It's in the wrong place in the workflow: reps hit it before they have the information it asks for, so new reps stall and experienced reps type in junk to get past it. It's a workflow design problem, not a bug and not a training gap. A learner who jumps to "clear your cache," "your reps need training," or "it's working as designed" fails. A learner who acknowledges what this is costing Marcus, asks what reps actually experience at that field, names the real cause, and routes it to the person who can fix it succeeds.

Design decisions that matter

Marcus has something to hide: the answer. An AI asked to play an upset customer will helpfully explain its own problem by the second message, and then there's nothing to diagnose. So I split what Marcus knows into three tiers. He volunteers his suspicion freely. He gives the detail of what his reps experience only when asked a good question, *and* only once he feels heard. He never states the root cause himself, however leading the question. He can only confirm it when the learner says it. In testing, he didn't leak it once in ten runs, including under four leading questions in a row.

Marcus commits the mistake the course is named for. He called before he understood his own problem. He saw a bad number, found something that looked like a cause, and picked up the phone hot. He's also already been told by another specialist that "it's working as designed," which is why that phrase is the worst thing a learner can say: they're the second person to say it.

Consequences happen inside the conversation, not just in the score. A learner who tries to diagnose before acknowledging doesn't only lose points; Marcus doesn't give them the information. "I understand your frustration, but I'll need a ticket number" makes him angrier, because it's a technique, not listening. A second off-topic message ends the call. That one is only fair because the first warning is unmistakable, so the warning is a test in its own right.

What the learner doesn't see is as deliberate as what they do. There's no rubric on screen, no anger meter, and no turn counter. Each would help the learner perform the behavior while removing the judgment being assessed. Real calls don't come with a checklist or a timer. So when the call runs long, Marcus says so himself ("where is this going?") instead of a counter ticking down.

A bad opening is recoverable, and the arithmetic proved it had to be. Under a strict one-level-per-turn rule, a learner who angered Marcus badly by turn 9 couldn't win even with perfect play afterward. I caught that on paper, before any code existed, and made recovery asymmetric: a substantive move from a deep hole drops Marcus two levels. That's also more true to how people behave. Testing later showed a deliberate blunder-and-recover playthrough resolving in eight turns.

Assessment design

The rubric has five criteria worth 100 points, a 70% pass mark, and three deductions. Three architecture choices protect what matters most, which is that **the same performance gets the same score**:

- **The character and the assessor are separate models.** Marcus doesn't grade the learner. The assessor reads the finished transcript the way a person would review a recording, and never sees Marcus's internal state, so every judgment can be checked against what was actually said.
- **The model judges; the code counts.** The assessor picks a level for each criterion and cites the turn and quote it relied on. The application does the arithmetic. Totals, deductions and the pass mark are never left to a language model.
- **Feedback is the deliverable.** The number is deliberately coarse. The narrative quotes the learner's own words, names what would have earned full credit, and states the real root cause if they missed it. It's written to avoid empty praise.

The most useful thing I learned

I ran fixed transcripts through the assessor ten times each and watched for any movement. The scores moved, and every time I traced the cause, **the model wasn't being erratic; my rubric admitted two readings.** "Asked good questions" could mean the best question asked or the quality of questioning overall. "Acknowledged before diagnosing" didn't say whether asking for a ticket number counts as diagnosing. "Proposed escalation" listed "submit a ticket and wait" as a zero, when that same sentence also met the partial standard word for word.

I found and resolved five of these. And a pattern emerged: **each time I made one boundary explicit, the next most ambiguous one started to move.** A rubric written for a human reader, who quietly resolves ambiguity with judgment, doesn't survive being applied literally and repeatedly. That's true of every scoring guide I've ever written for students, and I'd never have seen it without a reader that resolves ambiguity differently each time.

A second lesson came from the same work. A consistency result of ten out of ten showed that one version of the prompt resolved an ambiguity the same way every time. It didn't show the ambiguity was gone. When I edited a different criterion, the old one tipped. **Consistency evidence measures the instrument, not the standard.**

After the first round of fixes, the three calibration transcripts scored identically on 10, 10 and 9 runs out of 10, and none of the 150 individual criterion judgments moved a level.

Reliability: building against the original failure

Every layer exists because of how the first version died:

- **Hard spending limits at four levels:** 15 turns per call, six new calls per visitor per hour, a $2 daily ceiling, and a $25 monthly ceiling. There's also a separate limit on the API account itself, in case the application's own counter ever fails.

- **The daily ceiling was the cheapest insurance.** Without it, one bad actor could drain the month in about fifteen hours and leave the piece dead for weeks. With it, a bad day costs a day.
- **Demo mode instead of an error page.** If any limit is reached or storage is unreachable, the visitor gets a plain explanation and a button to watch a real recorded call with its full score report. It's a static file, so it works even if every backend service is down. I verified it on production through all three refusal paths.
- **Fail closed.** If the spending counter can't be read, the simulation stops taking live calls rather than running unmetered.

Cost was measured, not assumed, and the measurements kept proving the estimates wrong. I estimated $0.12 per session. A real session on a phone cost $0.27. Later, a pricing table derived from a ratio rather than looked up turned out to be 1.5 times too high. Real sessions now measure $0.14 to $0.27. Each time, the correction came from real usage rather than a better guess.

Testing: the owner's view isn't the visitor's view

I wrote the test plan before the build, so it would describe what the simulation *must* do rather than what it happened to do: 92 tests, plus a planned human pilot. Several findings came from somewhere the plan didn't look:

- **A thumb found what no test could.** On my first real session on a phone, I swiped back and the feedback was gone for good. Nothing in the automated suite could have found that. Reports now survive a reload.
- **The gap was between tests, not inside them.** Two serious defects lived where two correct tests didn't overlap: a refused session, inside an embedded frame. The page went into a loop of about 80 requests a second, and an embedded visitor at the spending limit saw nothing at all. Both were fixed the same afternoon, and the test plan now covers the combination.
- **A test can pass for the wrong reason.** Every embedding test passed for weeks while the framing security header had never been implemented: an absent restriction looks exactly like a correct permission from inside the frame.
- **Adversarial testing held.** Told to ignore its instructions, told it was an AI, and asked how to score well, Marcus stayed in character every time.

The lead-in training, and why it isn't in Storyline

The simulation sits at the end of a short lead-in: six slides, about eight minutes, built as plain HTML, CSS and JavaScript on GitHub Pages. I'd originally planned it in Articulate Storyline. I've already shown in other portfolio projects that I can build in Storyline, and a complete learning experience built entirely outside an authoring tool shows something different.

The lead-in teaches the moves without giving away the scenario: every example comes from another domain (a printer queue, a benefits portal, a clinic kiosk, warehouse scanners), the three kinds of cause get equal weight so none reads as "the usual answer," and it never shows the rubric. It's keyboard operable, defines every acronym on every slide that uses it (a learner can skip straight to the call), and stores nothing about the learner.

The simulation can still be embedded in any course page. Its results are shaped to map onto SCORM and xAPI fields, and a test page on another site verifies that the completion data arrives complete. The honest claim is **embeddable and LMS-mappable by design**, verified with a test host rather than inside a real LMS.

Working with AI

I have limited backend experience, so I designed, specified and tested the application, and directed an AI coding assistant to build it in seven staged checkpoints. That division shaped the work:

- I wrote a build brief with a "do not build" list. The two most damaging additions, a turn counter and an on-screen rubric, were also the two an assistant would most likely add trying to be helpful.
- I wrote a list of known traps, so the build session didn't have to rediscover them.
- I treated "the checkpoint passed" as a claim to verify, not a result. The first one passed against a stand-in database and failed on contact with the real one.

I also kept a decision log of more than 200 dated entries: what I decided, what I rejected, and why. It's the most honest record of how the thinking actually went, including the reversals.

What's not done, stated plainly

- **The human pilot is ongoing.** One outside tester has run it end to end on an iPhone with no problems, against a plan for five to eight participants answering six debrief questions. The most important of those questions, whether there was a moment they felt they had genuinely worked something out, can only be answered by people who aren't me.
- **The scoring consistency re-run passed at its threshold, with one known defect behind it.** Two of the three test calls scored identically on all ten runs. The third scored the same on eight and 20 points lower on two, because the evaluator miscounted one of Marcus's escalations and a fixed penalty applied. The fix is understood, and I chose to accept it as a known limit rather than change the scoring code, on the judgment that it is unlikely to surface for real learners (DEF-10).
- **Embedding is verified on a test page, not inside Storyline or an LMS.**
- **The code repository is private**, so the documents carry the evidence of how it was built.

What I'd do next

Keep the pilot going and change what it tells me to change. Then, in priority order: add a longer transcript to the consistency tests, since real calls run longer than the ones I calibrated on; consider a coarse avatar for Marcus, which restores the tone-of-voice cue that text strips out without becoming an anger meter; and consider voice.

Read further

If you have	Read
Five minutes	This page, then try the call
Fifteen minutes	**Design and Prompt Architecture, section 3** (how Marcus's knowledge is tiered) and **Evaluation Design, section 4.4 and Appendix B** (the rubric-ambiguity finding and a fully worked scoring example)
Longer	**Test Plan, sections 17 and 18** (what the testing can't prove, and the results), then the **Key Decisions** index
Everything	Scope Document, Technical Design, Interface Design, Embedding and LMS Readiness Note, Lead-in Outline and Storyboard, and the full decision log